One-to-one SkewGRAM: Applying NLP Embeddings to Biological Graph Optimization

Bechir Mady[1], Abderahmane Abderrahmane[1], Mohamedou Yahya Cheikh Mohamed Vall[1], El Moustapha Mohamed El Moustapha[1], Mohamed Lemine Ahmed Sidi[1], and Hafedh Mohamed Babou[2]

[1] Unité de Recherche en Systèmes Intelligents Avancés (URSIA), Institut Supérieur du Numérique (SupNum), Nouakchott, Mauritania
{22014,22053,25239,22013}@supnum.mr, m-lamine.ahmed-sidi@supnum.mr
[2] École Supérieure Polytechnique (ESP), Nouakchott, Mauritania
hafedh.mohamed-babou@esp.mr

Abstract. The One-to-one SkewGRAM problem asks for a longest path in a directed acyclic graph D whose vertices induce a connected subgraph in an undirected graph G defined on the same vertex set. It arises when metabolic pathways are compared with genomic or functional networks, and it is NP-hard. The exact ILP2 model needs tens of seconds on 100-vertex instances and minutes on 200-vertex ones, while earlier graph neural network decoders return paths of about one vertex. Our method, Skew-GRAM, borrows the Skip-Gram model from natural language processing: directed random walks on D are read as sentences, and vertex embeddings are learned with negative samples drawn from non-successors. A Monte-Carlo decoder combines these embeddings with an exact dynamic-programming lookahead on D and a bonus for vertices adjacent in G to the current path. A final Union-Find step extracts the longest sub-path that is connected in G, so every returned path is feasible. Over five random seeds on 368 Erdős–Rényi instances with an ILP2 solution, the method reaches the ILP2 path length in 71.3 % of the runs with 100 vertices and in 30.0 % with 200 vertices, with median speedups over ILP2 of 4.0× and 21.2×; on one 200-vertex instance it finds a longer path than the reported ILP2 solution. It finds the exact optimum on all 25 KEGG metabolic pathways and handles 1000-vertex graphs in about 143 seconds. Without the dynamic-programming lookahead, the paths are 23 % shorter on 100-vertex instances.

Keywords: Combinatorial optimization · Bioinformatics · Node embeddings · Skip-Gram · Dynamic programming · Union-Find.

1 Introduction

Biological systems are naturally described as networks whose vertices are biological entities and whose edges are relations between them. Comparing two networks built on the same entities can reveal patterns that neither network shows alone. Metabolism, for instance, is a directed network in which reactions

follow each other along pathways, while genomic proximity or protein–protein interactions give an undirected network of functional associations [15, 18]. A chain of reactions whose genes are also close to each other on the genome is a candidate for a conserved functional unit.

The *One-to-one SkewGRAM* problem formalizes this search [1, 8]. Given a directed acyclic graph D and an undirected graph G on the same vertex set, it asks for a longest directed path of D whose vertices induce a connected subgraph of G. The problem is NP-hard [3, 8]. Its difficulty comes from the interaction of the two constraints: a valid successor in D may break connectivity in G, and vertices that are close in G need not lie on a common path of D.

Exact methods solve the problem to optimality. The integer linear program ILP2 [1, 3] needs about 30 seconds on 100-vertex instances and about four minutes on 200-vertex instances of our benchmark. Learning-based approaches have also been studied [2]: a dual graph convolutional network with heuristic post-processing, and a graph encoder with an LSTM-attention decoder (DL2). Their sequential decoding produces very short paths, about 1.2 vertices on average for DL2.

This paper takes a different route, from natural language processing (NLP) to biology. Our method, Skew-GRAM, reads directed random walks on D as sentences and learn vertex embeddings with the Skip-Gram model [13], as DeepWalk and node2vec do for graphs [9, 14]. The embeddings alone do not solve the optimization problem, so we use them as one of three signals in a decoder designed for the problem, next to an exact dynamic-programming lookahead on D and a soft connectivity signal from G. An exact extraction step then enforces the connectivity constraint.

Our contributions are:

1. a Skip-Gram embedding model for the vertices of a DAG, trained on directed random walks with hub sub-sampling and structural negative sampling;
2. a guided decoder that combines an exact longest-path lookahead on D, embedding similarity and a G-connectivity bonus, followed by an exact Union-Find extraction that guarantees feasibility;
3. an evaluation on 375 synthetic instances with 100 and 200 vertices (368 with an ILP2 solution) over five random seeds, on 15 larger instances with 500 and 1000 vertices, and on 25 real metabolic pathways from KEGG [10] with exact optima, together with an ablation of the lookahead.

2 Related Work

The One-to-one SkewGRAM problem belongs to a family of problems that compare heterogeneous biological networks, such as the search for metabolic and genomic patterns in CoMetGeNe [18] and network comparison methods surveyed in [15]. Fertin et al. [8] studied its complexity and gave exact algorithms for it. Ahmed Sidi et al. proposed the integer linear programs ILP1 and ILP2 [1] and constraint programming models [3]; ILP2 is the exact reference used in this paper.

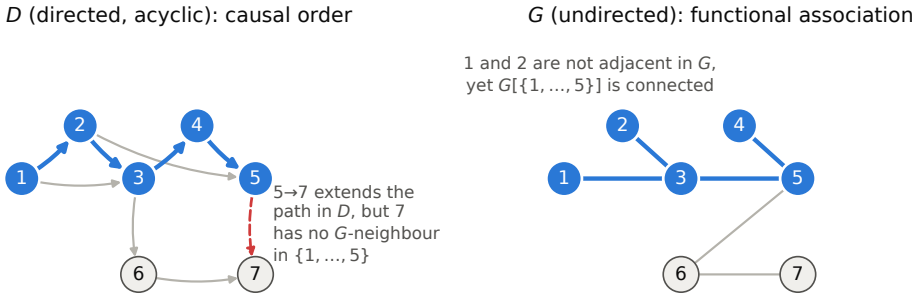


Fig. 1. Toy instance on seven vertices. The highlighted path $1 \to 2 \to 3 \to 4 \to 5$ is the unique optimum. Extending it by the arc $5 \to 7$ gives the longest path of D, but vertex 7 has no neighbour in G among the other path vertices. The prefix $(1, 2)$ is disconnected in G although the full path is connected.

Learning methods for combinatorial optimization on graphs include pointer networks [17], reinforcement learning of construction policies [5,6] and unsupervised training of graph neural networks [11]. These models are trained across many instances and then decode a solution. The deep learning approaches of [2] follow this scheme for our problem. Our model is trained on each instance separately, because vertex identifiers carry no shared meaning across independently generated graphs, and the learned embeddings only guide a search whose feasibility is enforced exactly. The embedding model itself follows word2vec [13] and its graph versions DeepWalk [14] and node2vec [9].

3 Problem Formulation

Let V be a set of n biological entities, $D = (V, A)$ a directed acyclic graph that models a causal or sequential relation, and $G = (V, E)$ an undirected graph that models functional association. A path $P = (p_1, \ldots, p_k)$ is (D, G)-*consistent* if

$$(p_i, p_{i+1}) \in A \quad (i = 1, \ldots, k-1) \qquad \text{and} \qquad G[\{p_1, \ldots, p_k\}] \text{ is connected.} \quad (1)$$

The goal is to find a (D, G)-consistent path with the largest number of vertices. Fig. 1 shows a small instance.

Connectivity cannot be checked incrementally. The connectivity constraint is global. Two consecutive vertices of a feasible path need not be adjacent in G, and a prefix of a feasible path can be disconnected in G while the full path is connected (Fig. 1). This is the common case: in our benchmark, all 368 ILP2 solutions contain two consecutive vertices that are not adjacent in G, and 363 of them (98.6 %) have a disconnected prefix. A decoder that rejects every extension which disconnects the current prefix could not build most of these solutions. Our decoder uses G only as a soft signal while it builds a path, and checks connectivity exactly once the path is complete.

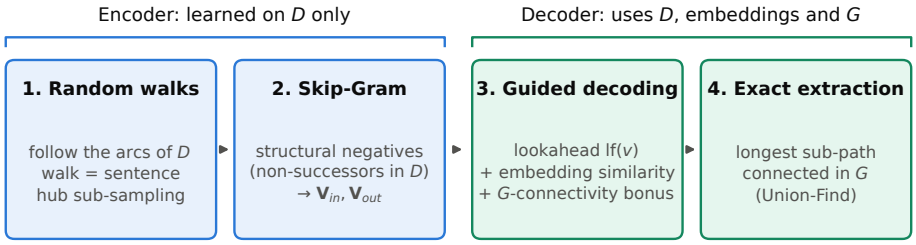


Fig. 2. The four phases of Skew-GRAM, run once per instance. The encoder learns embeddings $\mathbf{V}_{in}, \mathbf{V}_{out}$ from walks on D; the decoder generates candidate paths (Step A) and extracts a (D, G)-consistent sub-path from each of them (Step B).

4 Method

Skew-GRAM has an encoder, which learns vertex embeddings from D only, and a decoder, which uses D, the embeddings and G to build paths (Fig. 2). It runs once per instance in four phases. The guiding analogy is taken from NLP: a vertex plays the role of a word, a directed random walk the role of a sentence, and the neighbourhood of a vertex in a walk the role of its context.

Phase 1: random walks and hub sub-sampling. From every vertex with at least one successor, we start several random walks that follow outgoing arcs of D: from vertex u the next vertex is drawn uniformly from its successors $N^+(u)$. A walk stops at a sink or at a maximum length. As in word2vec, each occurrence of a vertex v with relative frequency $f(v)$ is kept with probability $\min\big(1, \sqrt{t/f(v)} + t/f(v)\big)$, $t = 10^{-3}$, which reduces the weight of hubs. A sliding window over each walk then yields (centre, context) pairs.

Phase 2: Skip-Gram embeddings with structural negatives. We learn two matrices $\mathbf{V}_{in}, \mathbf{V}_{out} \in \mathbb{R}^{n \times d}$. For a centre vertex c, a context vertex o and K negative vertices $n_1, \ldots, n_K$, the loss is the usual negative-sampling objective

$$\mathcal{L}(c, o, n_{1:K}) = -\log \sigma(\mathbf{v}_{in,c}^{\top} \mathbf{v}_{out,o}) - \sum_{k=1}^{K} \log \sigma(-\mathbf{v}_{in,c}^{\top} \mathbf{v}_{out,n_k}). \tag{2}$$

The difference with standard Skip-Gram lies in how negatives are drawn: instead of a frequency-based distribution, the negatives of c are drawn uniformly among the vertices that are not successors of c in D. The negative signal thus reflects the direction of the arcs. The decoder uses the similarity $\mathrm{sim}(u, v) = \mathbf{v}_{in,u}^{\top} \mathbf{v}_{out,v}$.

Phase 3: guided decoding (Step A). A decoder driven by similarity alone tends to pick successors that lead quickly to a sink. We therefore compute, for

every vertex v, the number $\mathrm{lf}(v)$ of arcs on a longest path of D that starts at v:

$$\mathrm{lf}(v) = 0 \text{ if } N^+(v) = \emptyset, \qquad \mathrm{lf}(v) = \max_{w \in N^+(v)} \big(1 + \mathrm{lf}(w)\big) \text{ otherwise.} \tag{3}$$

Because D is acyclic, all values are obtained in one pass over a reverse topological order, in $O(|V| + |A|)$ time [4]. When the current vertex is u and $v \in N^+(u)$ has not been visited yet, the decoder scores v by

$$S(u, v) = \frac{\mathrm{lf}(v)}{\tau_{lf}} + \beta \, \frac{\mathrm{sim}(u, v)}{\tau} + \log(\gamma) \, \mathbb{I}_G(v), \tag{4}$$

where $\mathbb{I}_G(v) = 1$ if v is adjacent in G to a vertex of the current path and 0 otherwise. The next vertex is drawn with probability $P(v \mid u) \propto \exp S(u, v)$. The three terms measure the remaining path potential in D, the learned compatibility between u and v, and a preference for vertices connected in G to the path built so far. Visited vertices are masked. The decoder repeats such walks W times from uniformly chosen start vertices.

Phase 4: exact extraction (Step B). A candidate path $P = (p_1, \ldots, p_L)$ from Phase 3 is a path of D but may be disconnected in G. Every contiguous sub-path of P is still a path of D, so it suffices to find the longest interval $[i, j]$ such that $G[\{p_i, \ldots, p_j\}]$ is connected. For each start i, we extend the window one vertex at a time and maintain the components of the window in a Union-Find structure [16], comparing each new vertex with the vertices already in the window. This costs $O(L^3 \alpha(L))$ for a path of length L, where α is the inverse Ackermann function. The extraction is only run when the raw candidate is longer than the best sub-path found so far, and L is bounded by the longest path of D (at most 100 vertices in our instances). The best extracted sub-path over the W walks is returned; by construction it is (D, G)-consistent.

5 Experiments

5.1 Setup

Synthetic instances. The main benchmark contains 300 instances with $n = 100$ vertices and 75 instances with $n = 200$ vertices, each provided with the solution of ILP2 (path and running time). D is built by drawing a random topological order and adding each forward arc with probability p ($p = 0.08$ for $n = 100$, $p = 0.05$ for $n = 200$), and G is an independent Erdős–Rényi graph $G(n, p)$ [7]. ILP2 reports no solution for 2 instances with 100 vertices and 5 with 200 vertices; these are excluded from all comparisons with ILP2, which leaves 298 and 70 instances. ILP2 is an exact model, but the reported solutions are not all optimal: on instance 200_68, Skew-GRAM finds a feasible path with 10 vertices, which we checked independently, while the ILP2 file reports 7. We therefore use the ILP2 solutions as reference lengths and say that a run reaches ILP2 when

Table 1. Hyperparameters, identical for all instances.

Encoder (Phases 1–2)		Decoder (Phases 3–4)		
Walks per start vertex	40	Decoding walks	W	30,000
Maximum walk length	30	Lookahead temperature	τ_{lf}	1.5
Context window	3	Similarity temperature	τ	0.5
Embedding dimension d	32	Similarity weight	β	0.3
Negatives per pair K	5	G-connectivity bonus	γ	6.0
Epochs / learning rate	8 / 0.01			

its path is at least as long. To study scalability we also use 10 instances with $n = 500$ and 5 instances with $n = 1000$, built in the same way with $p = 0.05$ (we generated the 1000-vertex instances ourselves); no ILP2 solution is available for them.

Real metabolic networks. We use 25 metabolic pathways from KEGG [10], taken from seven bacteria (KEGG codes aae, bsu, eco, mtv, pae, sau and syn). In each instance, D is the metabolic graph of a pathway and G a graph of genomic or functional proximity on the same vertices. The set contains 19 acyclic pathways (at most three per organism, with 10 to 80 listed entities) and 6 pathways whose only cycles are reversible reactions (2-cycles); vertices without any arc or edge are discarded on loading, which leaves 4 to 38 vertices. Each reversible reaction is oriented from the lower to the higher vertex identifier so that D is acyclic; this is a modelling simplification. These instances have no ILP2 solution, but they are small, so we compute their exact optimum by enumerating the paths of D with two exact cuts: the vertices of a feasible path lie in one connected component C of G, so only paths of $D[C]$ are enumerated, and a path ending at u cannot gain more than $\mathrm{lf}(u)$ vertices.

Protocol. Table 1 lists the hyperparameters, which are the same for all instances. Training uses Adam [12]. Each instance is solved with five seeds (42 to 46); the seed is reset at the start of each instance, so a result does not depend on the order in which instances are processed. Every returned path is checked to be (D, G)-consistent. Skew-GRAM runs on an Intel Core i5-1135G7 laptop processor (4 cores), with four instances solved in parallel and one thread each; the laptop was also used for other work during the runs, so times include some load-related noise. ILP2 times come from the dataset files and were measured on another machine, so speedups are indicative only.

5.2 Comparison with ILP2

Table 2 summarizes the results. Values after ± are standard deviations across the five seeds of the corresponding mean, except for running times, where they are standard deviations over all runs.

Table 2. Skew-GRAM compared with ILP2 on the synthetic instances with an ILP2 solution (five seeds per instance). ILP2 length reached: runs whose path is at least as long as the ILP2 solution. Speedup: median over instances of the ratio between the ILP2 time and the mean Skew-GRAM time.

	$n = 100$	$n = 200$
Instances with an ILP2 solution	298 / 300	70 / 75
ILP2		
Mean length (vertices)	9.93	9.64
Mean time (s)	29.1	251.8
Skew-GRAM		
Mean length (vertices)	9.47 ± 0.02	7.89 ± 0.12
Mean gap to ILP2 (%)	-4.4 ± 0.2	-17.2 ± 0.9
Runs reaching the ILP2 length (%)	71.3 ± 2.1	30.0 ± 4.8
Runs within one vertex of ILP2 (%)	90.3 ± 1.0	53.1 ± 3.3
Mean time (s)	6.9 ± 2.6	11.3 ± 4.2
Median speedup over ILP2	4.0×	21.2×

With 100 vertices, the paths found by Skew-GRAM have 9.47 vertices on average, against 9.93 for ILP2, a mean gap of −4.4 %. It reaches the ILP2 length in 71.3 % of the runs and is within one vertex of it in 90.3 %. A run takes 6.9 s on average, against 29.1 s for ILP2. With 200 vertices the problem is harder for the heuristic: the mean gap grows to −17.2 % and the ILP2 length is reached in 30.0 % of the runs. ILP2 slows down much more, from 29.1 s to 251.8 s on average, while Skew-GRAM goes from 6.9 s to 11.3 s, so the median speedup rises from 4.0× to 21.2× (Fig. 3b). With 100 vertices no run returns a path longer than ILP2; with 200 vertices, four of the five runs do so on instance 200_68 (Table 3). For reference, DL2 [2] was reported to find paths of 1.2 vertices on average on 50-vertex instances; we did not re-run it, so this figure is only indicative.

Table 3 details ten instances, chosen at the 0th, 50th, 80th, 95th and 100th percentiles of the mean gap for each size. With 100 vertices most instances reach the ILP2 length, and the gap is concentrated on a few instances on which the guided walks rarely reach the region of D that contains the best path. With 200 vertices the gaps are larger (up to 8 vertices in a run), while on instance 200_68 Skew-GRAM exceeds the ILP2 solution in four runs out of five.

Stability across seeds. The decoder is stochastic, so we measure how results vary with the seed. The mean length over all instances is very stable (standard deviation 0.02 vertex for $n = 100$ and 0.12 for $n = 200$). At the level of single instances, the length returned for an instance changes with the seed by 0.43 vertex on average (standard deviation) for $n = 100$ and 0.93 for $n = 200$; the five runs return the same length on 48 % and 23 % of the instances. The running time of an instance also varies between runs (standard deviation 2.4 s on average for $n = 100$), mostly because the machine was shared with other work: the

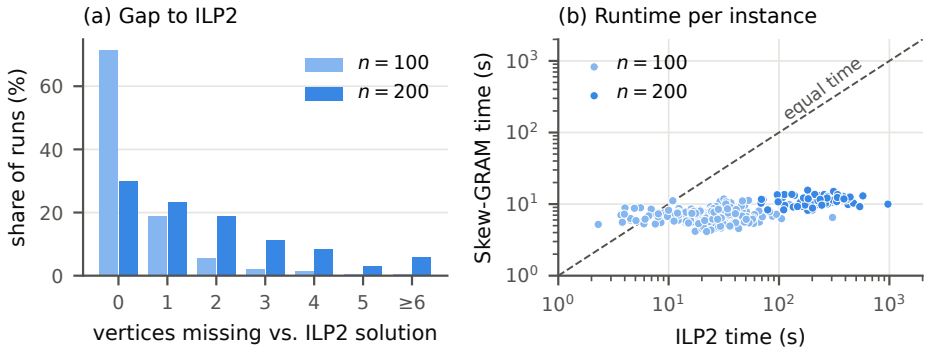


Fig. 3. (a) Number of vertices missing from the Skew-GRAM path compared with the ILP2 solution, over all runs (0: ILP2 length reached or exceeded). (b) Running time per instance (Skew-GRAM: mean over the five seeds); points below the dashed line are faster with Skew-GRAM.

amount of computation is the same in every run, since the number of walks is fixed. Keeping the best of the five runs would raise the number of instances on which the ILP2 length is reached to 273 of 298 and 42 of 70, at five times the cost.

5.3 Real Metabolic Networks

Table 4 reports the results on the 25 KEGG pathways. Skew-GRAM finds the exact optimum on every pathway and with every seed. These instances are much easier than the synthetic ones: they have at most 38 vertices, and G is very sparse (8.7 edges on average), so the optimum is short (3.4 vertices on average) and equals a single vertex for 7 pathways. Exact enumeration with the component cut takes less than 2 ms per pathway, far less than the 2.9 s of Skew-GRAM. On such small networks an exact method is the better choice; the experiment shows that the method applies unchanged to real data and returns correct solutions, but it does not demonstrate a speed advantage. Larger real networks, such as genome-scale metabolic networks coupled with protein interaction data, are needed to assess the method in the regime where exact solvers struggle.

5.4 Ablation of the DAG Lookahead

To measure the role of the lookahead, we remove the term $\mathrm{lf}(v)/\tau_{lf}$ from $S(u, v)$ and keep everything else unchanged (seed 42). With 100 vertices, the mean length drops from 9.49 to 7.29 vertices ($-23.2\,\%$), paths become shorter on 229 of 298 instances, and the ILP2 length is reached on 47 instances instead of 216. With 200 vertices, the mean length drops from 7.87 to 6.40 ($-18.7\,\%$) and the ILP2 length is reached on 5 instances instead of 24. Embedding similarity alone does not tell the decoder whether a successor leads to a long branch or to a sink; the lookahead provides this information exactly and at linear cost.

Table 3. Per-instance results for ten instances, chosen at the 0th, 50th, 80th, 95th and 100th percentiles of the mean gap for each size (best first). Skew-GRAM: mean ± standard deviation over the five seeds. A positive gap means a longer path than the ILP2 solution.

	Length (vertices)		Gap	Time (s)	
Instance	ILP2	Skew-GRAM	(%)	ILP2	Skew-GRAM
100_4	11	11.0 ± 0.0	0.0	29.0	6.1
100_230	10	9.8 ± 0.4	−2.0	16.1	8.2
100_133	13	12.0 ± 0.0	−7.7	20.6	6.4
100_247	6	5.0 ± 0.0	−16.7	27.0	9.3
100_300	9	5.0 ± 0.0	−44.4	16.9	7.9
200_68	7	9.2 ± 1.3	31.4	183.9	11.9
200_50	9	7.6 ± 1.3	−15.6	134.6	9.1
200_61	7	5.0 ± 0.0	−28.6	440.3	12.5
200_49	9	5.4 ± 0.9	−40.0	225.2	11.2
200_41	13	7.2 ± 1.6	−44.6	108.1	12.4

Table 4. Results on 25 KEGG metabolic pathways (five seeds each). LP(D): number of vertices on a longest path of D. Optimum: exact optimum by enumeration. Hits: pathways on which Skew-GRAM reaches the optimum (mean over seeds). Lengths are means.

Pathways	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert E\rvert$	LP(D)	Optimum	Skew-GRAM	Hits
Acyclic (19)	4–15	25.9	9.1	7.5	3.58	3.58	19/19
With 2-cycles (6)	12–38	97.3	7.3	13.5	2.83	2.83	6/6
All (25)	4–38	43.1	8.7	9.0	3.40	3.40	25/25

5.5 Scalability

Table 5 shows how the running time grows with n and where it is spent. Walk generation is negligible. Training grows with the size of the walk corpus, which grows with n and with the length of the walks. Decoding takes the largest share at every size, from 91 % of the time for $n = 100$ to 57 % for $n = 1000$: its cost is proportional to the number of walks W, to the length of each walk, which is bounded by the longest path of D, and to the number of successors scored at each step, and the G-bonus compares each candidate with the vertices already on the path. With $n = 1000$ (about 25,000 arcs), one run takes 143 s on average. No ILP2 solution is available for these sizes, but the number of vertices on a longest path of D, LP(D), is an upper bound on the optimum: the paths found contain 33.5 and 72.8 vertices on average for $n = 500$ and $n = 1000$, against 46.6 and 94.4 for this bound.

The 30,000 walks are the main cost, so we examine how quality depends on W. Because the first k walks of a run use exactly the same random numbers as a run with budget $W = k$, the best length after each block of 500 walks gives the

Table 5. Running time per phase and path length as n grows (all instances, five seeds). $|A|$ and $\mathrm{LP}(D)$ (vertices on a longest path of D) are means over instances; Length: mean $\pm$ standard deviation across seeds of the mean; Total: mean $\pm$ standard deviation over runs.

				Length	Time per run (s)			
n	Inst.	$\lvert A\rvert$	$\mathrm{LP}(D)$	(vertices)	Walks	Training	Decoding	Total
100	300	399	15.1	9.47 ± 0.02	0.02	0.6	6.2	6.8 ± 2.6
200	75	999	19.6	7.83 ± 0.15	0.06	2.6	8.6	11.3 ± 4.2
500	10	6185	46.6	33.52 ± 1.32	0.25	19.1	29.7	49.1 ± 8.6
1000	5	24974	94.4	72.76 ± 1.82	0.58	60.5	81.4	142.6 ± 13.2

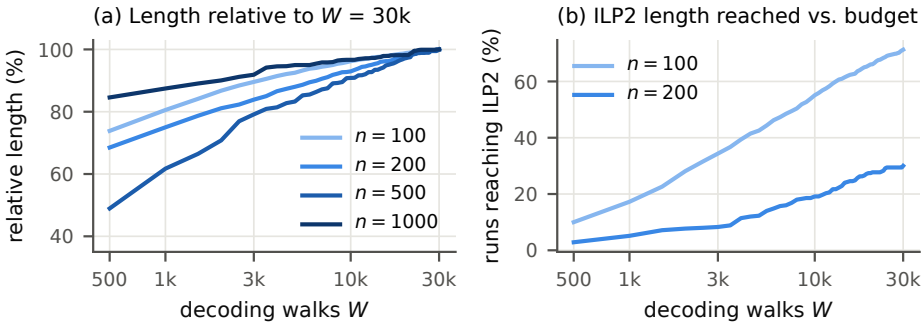


Fig. 4. Effect of the number of decoding walks W. (a) Mean path length relative to the length obtained with $W = 30{,}000$. (b) Share of runs that reach the ILP2 length (instances with an ILP2 solution).

result of every smaller budget at no extra cost (Fig. 4). Quality is still increasing at $W = 30{,}000$ for every size. With $W = 3{,}000$, the paths keep between 79 % ($n = 500$) and 92 % ($n = 1000$) of their final length, and the share of runs reaching the ILP2 length falls from 71.3 % to 34.4 % for $n = 100$. Reducing W is therefore not a free saving. Three remedies follow from the structure of the decoder. The walks are independent and can run in parallel. The G-bonus can be maintained incrementally by storing the set of G-neighbours of the current path, which makes it $O(1)$ per candidate instead of $O(L)$. Finally, the budget could adapt to the instance, for example by stopping when the best length has not improved for a given number of walks. We leave these engineering improvements to future work.

5.6 Discussion and Limitations

The results support the combination of learned and exact components. The lookahead gives the decoder exact information that is cheap to compute on a DAG, instead of asking the embedding model to learn it. G is used for guidance and for the final check rather than inside the embedding objective, which keeps

the learning problem focused on the order of the path, while the extraction step guarantees feasibility.

The method has clear limits. It is a heuristic and should not replace ILP2 when optimality must be guaranteed. Its gap to ILP2 grows with n for a fixed budget, and on small real pathways exact enumeration is both exact and faster. The embeddings are trained for each instance, which adds a training cost that a model trained across instances would avoid. Finally, our real-data evaluation covers small pathways only, and the orientation of reversible reactions is a simplification.

6 Conclusion

We presented Skew-GRAM, which applies the Skip-Gram model from NLP to the One-to-one SkewGRAM problem from biology: finding a longest path in a DAG whose vertices are connected in a second graph. Embeddings learned from directed random walks guide a Monte-Carlo decoder together with an exact DAG lookahead and a connectivity bonus, and an exact Union-Find step makes every returned path feasible. On 368 synthetic instances with an ILP2 solution, the method reaches the ILP2 length in most runs with 100 vertices and runs much faster than ILP2, with a speedup that grows with the size of the graphs; its average results vary little across seeds, it finds the exact optimum on 25 real metabolic pathways, and it scales to graphs with 1000 vertices. Future work will address larger real networks, a faster decoder (parallel walks, incremental G-bonus, adaptive budget) and embeddings trained across instances.

References

1. Ahmed Sidi, M.L., Bocquillon, R., Mohamed Babou, H., Dhib, C., Néron, E., Soukhal, A., Nanne, M.F.: Improved approaches to solve the one-to-one SkewGraM problem. Computers & Operations Research **138**, 105584 (2022)
2. Ahmed Sidi, M.L., Mohamed Babou, H., Dhib, C.: Deep learning approaches for finding a longest DG-consistent path in biological networks. Preprint (2024)
3. Ahmed Sidi, M.L., Mohamed Babou, H., Dhib, C., Néron, E., Soukhal, A., Nanne, M.F.: Constraint programming approaches for finding conserved metabolic and genomic patterns. Computers & Operations Research **183**, 107166 (2025)
4. Bellman, R.: On a routing problem. Quarterly of Applied Mathematics **16**(1), 87–90 (1958)
5. Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S.: Neural combinatorial optimization with reinforcement learning. arXiv preprint arXiv:1611.09940 (2016)
6. Dai, H., Khalil, E.B., Zhang, Y., Dilkina, B., Song, L.: Learning combinatorial optimization algorithms over graphs. In: Advances in Neural Information Processing Systems, vol. 30 (2017)
7. Erdős, P., Rényi, A.: On random graphs I. Publicationes Mathematicae Debrecen **6**, 290–297 (1959)
8. Fertin, G., Mohamed Babou, H., Rusu, I.: Algorithms for subnetwork mining in heterogeneous networks. In: Experimental Algorithms (SEA 2012). LNCS, vol. 7276, pp. 184–194. Springer (2012)

9. Grover, A., Leskovec, J.: node2vec: scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864 (2016)
10. Kanehisa, M., Goto, S.: KEGG: Kyoto Encyclopedia of Genes and Genomes. Nucleic Acids Research **28**(1), 27–30 (2000)
11. Karalias, N., Loukas, A.: Erdős goes neural: an unsupervised learning framework for combinatorial optimization on graphs. In: Advances in Neural Information Processing Systems, vol. 33 (2020)
12. Kingma, D.P., Ba, J.: Adam: a method for stochastic optimization. arXiv preprint arXiv:1412.6980 (2014)
13. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J.: Distributed representations of words and phrases and their compositionality. In: Advances in Neural Information Processing Systems, vol. 26 (2013)
14. Perozzi, B., Al-Rfou, R., Skiena, S.: DeepWalk: online learning of social representations. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710 (2014)
15. Sharan, R., Ideker, T.: Modeling cellular machinery through biological network comparison. Nature Biotechnology **24**(4), 427–433 (2006)
16. Tarjan, R.E.: Efficiency of a good but not linear set union algorithm. Journal of the ACM **22**(2), 215–225 (1975)
17. Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. In: Advances in Neural Information Processing Systems, vol. 28 (2015)
18. Zaharia, A., Labedan, B., Froidevaux, C., Denise, A.: CoMetGeNe: mining conserved neighborhood patterns in metabolic and genomic contexts. BMC Bioinformatics **20**, 19 (2019)